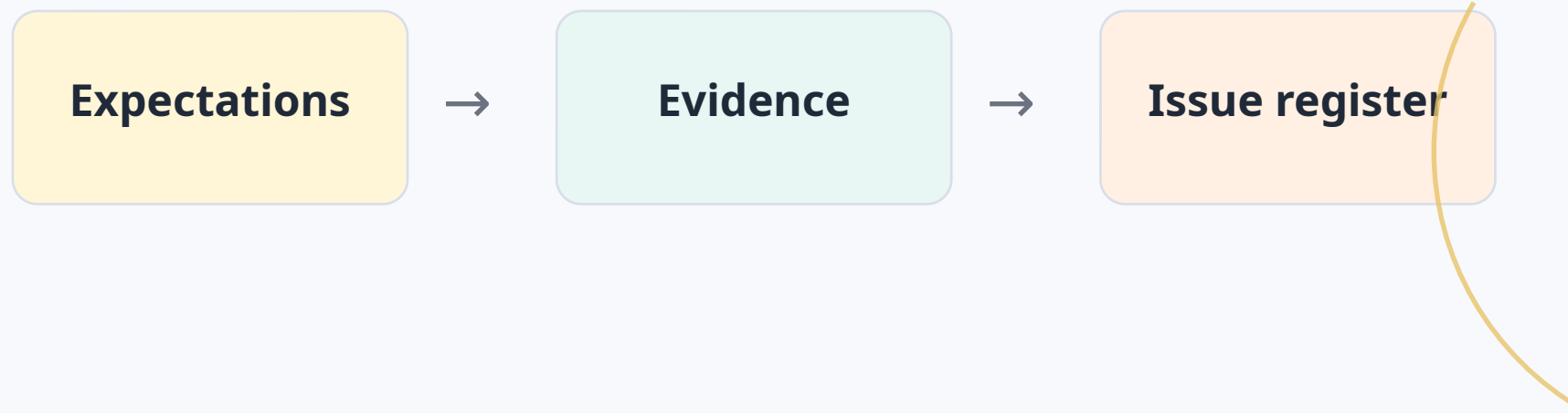


Inspecting Raw Datasets with Python and Pandas

Diagnose first. Clean later.



Today's learning path

1

Recognise raw-data problems

Read a data dictionary

3

Begin with broad inspection

4

Measure missingness and duplicates

5

Inspect values and conversions

6

Document an issue register

By the end, you should be able to inspect a raw dataset systematically, interpret Pandas output and record evidence without altering the source.

Inspection is diagnosis

Inspection asks whether the dataset is suitable for the intended analysis. It produces evidence; it does not silently repair the source.

Expect

Use context, a data dictionary and business rules to define what should be present.

Observe

Use Pandas methods to report structure, types, completeness, values and distributions.

Record

Describe the issue, evidence, consequence, severity and required follow-up.

Key boundary: conversion tests may create temporary Series, but the raw DataFrame remains unchanged.

The data dictionary sets expectations

Meaning

What does the field represent, and what is the unit of observation?

Rules

What type, format, range, categories and missingness are allowed?

Field	Expected type	Missing?	Allowed / range	Why it matters
StudentID	integer	No	unique positive ID	record identity
Mark	numeric	Yes	0–100 or documented code	valid summaries
ResultStatus	category	No	Pass / Fail / Absent	logic checks

A method output becomes evidence only when it is interpreted against documented expectations.

Typical raw-data issue families

Missingness

blank values and hidden sentinels

Duplicates

exact rows and repeated keys

Types

numbers or dates stored as text

Categories

case, spelling and whitespace variants

Validity

impossible ranges or invalid formats

Logic

contradictory fields and scope errors

Also inspect outliers, structural metadata, scale and privacy: not every risk appears inside a cell.

Missing values are not one simple problem

Visible

Blank CSV fields usually import as NaN or NA.

Hidden

-999, Unknown, N/A or a blank string may encode missingness without being recognised.

Cause

Non-response, failed capture, join loss or a field that is not applicable.

Impact

Counts, means, comparisons and model inputs can become biased or misleading.

Do not replace missing values until you understand why they are missing and whether the field is required.

Duplicates require a defined key

Repeated rows or identifiers may inflate totals, double-count people or create conflicting facts. Context determines whether repetition is valid.

Exact row

Every column matches. Use `raw.duplicated()` to flag later copies.

Business key

A StudentID repeats, but other fields differ. Inspect all members of the group.

Legitimate

One student may have many assessment rows in a long dataset.

Ask first: what combination of columns should uniquely identify one record?

Representation problems hide meaning

Mixed types

72 and Absent in Mark

Percentages

85 and 85% in one column

Dates

2026-02-01 and 01/02/2026

Currency

1200, R1200 and 1,200.00

Categories

Diploma IT, Dip IT, diploma it

Whitespace

north and north look different

One incompatible value can force a whole numeric column to import as text-like, hiding it from numeric summaries.

Validity, outliers and contradictions

Range

Mark = 105 violates a documented 0–100 rule.

Format

2026-02-30 cannot be a valid calendar date.

Outlier

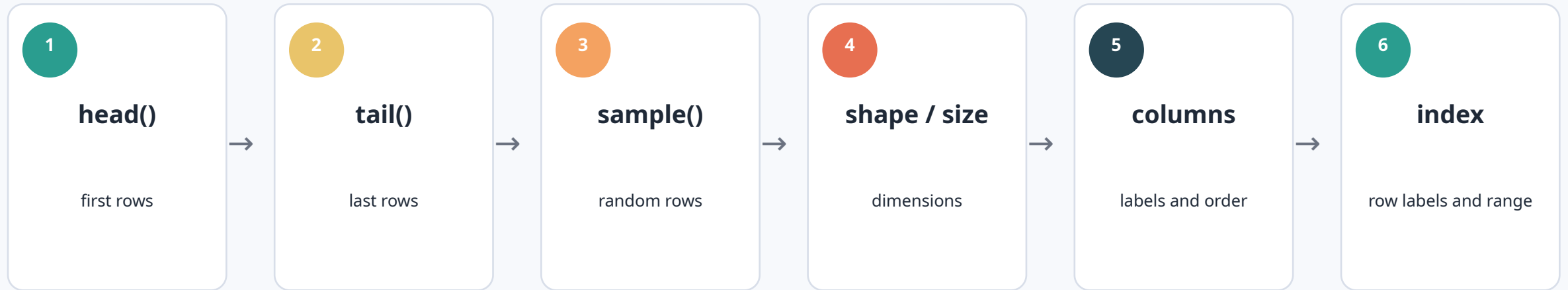
An extreme value is unusual, but may still be genuine.

Cross-field logic

Mark = 35 with ResultStatus = Pass is contradictory.

A violated business rule is direct evidence; an outlier is a prompt to investigate, not an automatic deletion instruction.

Begin broad: complementary views



No preview proves a dataset is clean. Broad views generate clues for targeted tests.

dtypes and info(): types, counts and scale

```
raw.dtypes
```

```
raw.info(  
    show_counts=True,  
    memory_usage="deep"  
)
```

dtypes

Reports one inferred dtype per column; unexpected text-like types are clues.

non-null counts

Compare each count with the row count to detect missingness.

index + columns

info() reports the index range, column names and column count.

memory

Deep memory usage helps assess text-heavy data and processing scale.

describe(): distribution summaries

Numeric columns

count, mean, standard deviation, quartiles, minimum and maximum.

Statistic	Reports	Possible clue	Limitation
count	non-missing n	missingness	not validity
mean / std	centre / spread	unusual distribution	sensitive to extremes

Text / category columns

count, unique, top and frequency when included.

Statistic	Reports	Possible clue
count	non-missing n	missingness
unique	distinct count	fragmented labels
top / freq	mode and count	dominant category

Use describe(include='all') for breadth. NaN in non-applicable statistic cells is not proof that the source value is missing.

Measure missingness by column and row

```
missing_count = raw.isna().sum()
missing_pct = raw.isna().mean().mul(100)
rows_with_missing = raw.isna().any(axis=1)
raw.loc[rows_with_missing]
```

isna()

Returns a Boolean mask with the same shape as the dataset.

sum()

Counts missing cells per column because True behaves like 1.

mean()

Reports the proportion missing; multiply by 100 for a percentage.

any(axis=1)

Flags records with at least one recognised missing value.

Inspect duplicates: rows and keys

```
exact = raw.duplicated(keep=False)
key_dupes = raw.duplicated(
    subset=["StudentID"],
    keep=False
)
raw.loc[key_dupes].sort_values("StudentID")
```

duplicated()

Returns one Boolean per row; the default flags later copies only.

keep=False

Flags every member of a duplicate group for inspection.

subset=...

Checks a documented business key instead of the full row.

Display rows

Counts alone cannot reveal whether repeated keys conflict.

Inspect categories and hidden variants

```
raw["Programme"].value_counts(dropna=False)
raw["Programme"].unique()
raw["Programme"].nunique(dropna=False)
[repr(v) for v in raw["Programme"].unique()]
```

value_counts()

Reports frequencies; dropna=False keeps missing values visible.

unique()

Shows distinct raw values in order of appearance.

nunique()

Reports cardinality; unexpectedly high counts suggest fragmentation.

repr()

Makes leading or trailing spaces and escape characters easier to see.

Temporary conversion is a diagnostic test

```
mark_num = pd.to_numeric(raw["Mark"], errors="coerce")
date_test = pd.to_datetime(
    raw["SubmissionDate"],
    errors="coerce", format="mixed", dayfirst=True
)
raw.loc[raw["Mark"].notna() & mark_num.isna(), "Mark"]
```

errors='coerce'

Unparseable values become missing in the temporary result.

Compare masks

Raw non-missing + converted missing isolates conversion failures.

Preserve evidence

Do not overwrite the raw column during inspection.

Interpret

A failure can be an error, a legitimate code such as Absent, or an ambiguous format.

Targeted rules and cautious outlier flags



Rules come from the data dictionary; statistical flags only identify unusual observations.

From output to an issue register

Record these fields

- Issue category
- Column(s) or key
- Reproducible evidence
- Why it is a problem
- Possible effect
- Severity
- Recommended follow-up

Quality test

- Can another analyst reproduce the count or row selection?
- Is evidence separated from assumptions?
- Is the consequence explained?
- Does follow-up request verification rather than inventing a correction?

A good register turns scattered console output into an auditable plan for the next cleaning lesson.

Practical: inspect student_results_raw.csv

Activity 1

Read the dictionary; state the unit of observation and expected rules.

Activity 2

Run broad checks: preview, shape, labels, dtypes, info and describe.

Activity 3

Measure missingness and investigate exact and key duplicates.

Activity 4

Inspect categories, conversion failures, ranges, scope and logic.

Activity 5

Submit the completed issue register with reproducible evidence.

Do not clean the dataset. Your deliverable is an evidence-based diagnosis.

What students should remember

Inspection is diagnosis; cleaning is a later decision.
Expectations come from context, a data dictionary and business rules.
Use broad methods first, then targeted tests.
Interpret what each method reports—and what it cannot prove.
Preserve the raw DataFrame and record reproducible evidence.
Next lesson: plan and apply justified cleaning decisions.

Final question: Which finding would you verify with the data owner first, and why?